The system, for people who will change it.

One Bun process, one SQLite file, one SolidJS client. The server owns the sessions, the tools, the permissions and the data; the browser is a view. This is the map, the request path, the extension points, and the gaps.

The request path

```
browser → fetch /session/:id/message → HttpApi (20 groups, packages/api) → SessionPrompt → ToolRegistry
(one registry per session) → tool (read/grep/edit/bash/…) → Provider → model → events back over SSE →
SessionProcessor persists parts → SQLite
```

LAYERS, NOT A CONTAINER

Every service is an Effect layer. `LayerNode.make(layer, deps)` declares one, and `buildLayer(graph)` assembles the graph with a cycle check. 56 services live in the server package; the HTTP application graph is a single group that the routes, the task runner and the task generator all share — so an in-server worker has exactly the context a request handler has.

Providers are resolved per turn, not per process: a model id is looked up in the catalog, and **DiveeComposer** resolves to the concrete `provider/model` it routed to. The synthetic model carries the target's capabilities, so the UI's attachment buttons match the model that actually answers.

DATA

26 tables, 46 migrations, one file. Sessions and messages exist twice — a v1 shape (`session_message`, `session_input`) and a v2 event shape (`session_message`, `session_context_epoch`) — with the v2 epoch table holding the context snapshot a session resumes from. Tasks add three: `agent_task`, a call journal (`agent_task_call`) and a hook audit trail (`agent_task_event`).

The packages

PACKAGE	HOLDS	NOTES
server	HTTP, sessions, tools, providers	343 files, 56 services
db	schema, migrations, providers, engine	344 files; the SQLite bridge
api	HttpApi definitions	20 groups; schemas are the contract
app	the SolidJS client	Vite; the v2 redesign is in progress
ui	the component library	120 components, 55 of them v2
llm	Schema-first LLM client	the provider boundary
plugin / plugins	the plugin SDK and bundled plugins	hooks, tools, config
memory / mcp / identity / sdk	the rest	graph memory, MCP host, brand, clients

Running it, extending it, and the gaps.

RUNNING IT

- **One process.** `bun run dev:both` starts the server (`:4097`) and the client. The server serves the built UI too.
- **Config precedence** is global → project → environment. The file is `diveeoi.json` (or `opencode.json`); both names are read, and the `diveeoi` name wins.
- **The database** defaults to `~/.local/share/diveeoi/diveeoi.db`; override with `DIVEEOI_DB`. Migrations apply on boot.
- **Auth is a password**, served by the server and checked per request. No OAuth, no accounts.
- **Tests:** 2 688 server, 415 client, 930 database. Typecheck per package with `tsgo`.

BACKGROUND TASKS

A task is a session with a schedule. The runner polls two queues — tasks whose cron is due, and tasks left `queued` — runs them through the same prompt path an interactive session uses, and records the outcome. A call journal makes a crash recoverable: committed calls are replayed verbatim, a call that was in flight is surfaced as "interrupted, verify before retrying".

The agent drives them too, through `task_create`, `task_list`, `task_status`, `task_logs` and `task_control`. A task can ask the generator agent to write its own spec first — prompt, schedule, agent, models — and then runs unattended.

EXTENSION POINTS

- **Agents** — `.opencode/agent/*.md` with frontmatter (model, mode, permission, tools).
- **Commands** — `.opencode/command/*.md`, with `$ARGUMENTS` and `$1`.
- **Skills** — `SKILL.md` folders, frontmatter with a name and a trigger description.
- **Plugins** — a `Plugin` export; hooks for config, events, tool execution, permissions.
- **MCP servers** — local or remote, declared in config; their tools join the registry.
- **Permissions** — per tool and per pattern, evaluated last-match-wins.

THE GAPS, WITH THEIR COST

GAP	WHAT IT MEANS
v2 redesign mid-migration	some surfaces are new components, some are the old ones
No TUI	browser only; there is no terminal client
One shared password	fine on a LAN, not a multi-user identity system
No built-in TLS	terminate at your own proxy; protect the DB file
SQLite concurrency	one writer at a time; fine for one operator
Free-model ceiling	the bundled tier is free and below a frontier model at hard reasoning
Model churn	the free tier moves; a model that fails is benched for a day